FIC–NEVPT2 解析梯度中的 CI 源项收缩优化
推导、实现与前后性能对照

基于本地代码和实测数据的学习笔记

2026 年 10 月 18 日

摘要

本笔记说明如何将原来逐 CI 系数生成过渡高阶密度矩阵的源项计算，改写为压缩多电子湮灭算符对整个 CI 向量的作用。旧算法完整保留为 `legacy` 后端，新算法为 `contracted` 后端。这是一项精确的代数重组，不改变 FIC–NEVPT2 的能量定义、CI 空间或收缩近似。本文件只提供 LaTeX 源代码，本次未编译。

目录

1 优化的是梯度的哪一部分 **1**

2 旧实现：逐 CI 单位向量求过渡 RDM **2**

3 从正常序 RDM 到湮灭后的 CI 向量 **2**

4 自旋分块和反对称轨道组合压缩 **2**

4.1 稀疏湮灭算符 3

5 直接计算整个 CI 源项 **3**

5.1 与 NEVPT2 代码的对应 4

6 如何切换并保留旧实现 **4**

7 测试条件和结果一致性 **4**

7.1 前后耗时 5

7.2 为什么完整梯度加速小于源项加速 5

8 完整结构优化前后对照 **6**

9 时间和内存标度仍有哪些限制 **6**

10 复现、回归与适用边界 **7**

11 学习时建议核对的三件事 **7**

1 优化的是梯度的哪一部分

设参考波函数为

$$|\Psi\rangle = \sum_{I=1}^{D} c_I |I\rangle, \qquad c^{\mathsf{T}} c = 1.$$

NEVPT2 梯度代码已经将微扰 Lagrangian 对 RDM 的导数累积为权重

$$W^{(k)} = \frac{\partial \mathcal{L}}{\partial \Gamma^{(k)}}, \qquad k = 1, 2, 3, 4.$$

在源代码中，它们分别保存在

```
self.rdms.ca
self.rdms.ccaa
self.rdms.cccaaa
self.rdms.ccccaaaa
```

需要计算的 RDM 部分 CI 源项为

$$b_I^{\mathrm{CI,RDM}} = \sum_{k=1}^{4} W^{(k)} : \frac{\partial \Gamma^{(k)}}{\partial c_I}. \tag{1}$$

这里 $W^{(k)}$ 是当前参考点上已经求出的偏导权重；应用链式法则时不应再对这些权重重复求导。这并不意味着完整 Lagrangian 就等于 $\sum_k W^{(k)} : \Gamma^{(k)}$；该表达式只是用于获得式 (1) 的辅助收缩。

本次只替换式 (1) 的计算。Fock/轨道能量带来的额外 CI 源项、其他态的贡献、半正则轨道响应、CI 归一化投影和 Z-vector 求解仍沿用原流程。

2 旧实现：逐 CI 单位向量求过渡 RDM

对每个 I 构造 e_I，再计算 $\langle I|\hat{O}^{(k)}|\Psi\rangle$。实数波函数下还需加上 bra/ket 交换的贡献：

$$\frac{\partial \Gamma^{(k)}_{PQ}}{\partial c_I} = \langle I|\hat{O}^{(k)}_{PQ}|\Psi\rangle + \langle \Psi|\hat{O}^{(k)}_{PQ}|I\rangle.$$

原实现的基本结构为

```
for I in range(state_ci.size):
    unit_ci.flat[I] = 1.0
    transition_rdms = make_tran_rdm1234(unit_ci, state_ci, ...)
    # Add the bra/ket transpose and contract with RDM weights.
    source.flat[I] += weighted_transition_contractions
    unit_ci.flat[I] = 0.0
```

因此会进行 D 次过渡 1–4-RDM 构造。总时间可以写成

$$T_{\mathrm{old}} \sim D\, T_{\text{transition RDM}}(a, D) + D\, T_{\text{RDM contraction}}(a),$$

其中 a 为活性轨道数。旧算法不一定同时保存 D 个 4-RDM；主要问题是重复生成、补全、转置和收缩。

3 从正常序 RDM 到湮灭后的 CI 向量

令 $P=(p_1,\dots,p_k)$、$Q=(q_1,\dots,q_k)$，自旋模式为 $\sigma=(\sigma_1,\dots,\sigma_k)$。正常序、自旋求和的 RDM 为

$$\Gamma^{(k)}_{PQ}=\sum_{\sigma}\langle\Psi|a^\dagger_{p_1\sigma_1}\cdots a^\dagger_{p_k\sigma_k}a_{q_k\sigma_k}\cdots a_{q_1\sigma_1}|\Psi\rangle.$$

定义湮灭后的向量

$$|T_{Q\sigma}\rangle=a_{q_k\sigma_k}\cdots a_{q_1\sigma_1}|\Psi\rangle.$$

对剩余电子的行列式 $|J\rangle$ 展开，有

$$\Gamma^{(k)}_{PQ}=\sum_{\sigma,J}T_{P\sigma,J}T_{Q\sigma,J}.$$

于是 RDM 可以视为湮灭后 CI 向量的 Gram 收缩，无需为每个原始 CI 单位向量单独计算过渡 RDM。

4 自旋分块和反对称轨道组合压缩

对 $k=m_\alpha+m_\beta$，把相同自旋计数的模式归为一块。每块只保存递增的同自旋轨道组合，维数为

$$M_s=\binom{a}{m_\alpha}\binom{a}{m_\beta},\qquad R_s=\binom{a}{n_\alpha-m_\alpha}\binom{a}{n_\beta-m_\beta}.$$

M_s 是压缩轨道元组数，R_s 是湮灭后 CI 空间的维数。超出实际电子数的块直接跳过。

设 $\widetilde{W}_s$ 为所有相应自旋排列和费米反对称符号合并后的权重。**使用组合选基不等于丢弃排列贡献**：代码显式把所有排列按正负号累积到 $\widetilde{W}_s$，因此不应再额外手动乘除 $k!$。

例如 $a=8,k=4,m_\alpha=m_\beta=2$ 时，

$$a^4=4096,\qquad M_s=\binom{8}{2}^2=784.$$

完整四体权重仍有 a^8 元素，但该自旋块的作用可在压缩空间中进行。

4.1 稀疏湮灭算符

构造

$$A_s\in\mathbb{R}^{(M_sR_s)\times D},\qquad T_s=\mathrm{reshape}(A_sc,M_s,R_s).$$

A_s 只包含允许的占据数连接和 ±1 的费米符号，采用 CSR 存储，不构造这个矩形矩阵的稠密表示。本项目 CI 下标按 alpha 字符串优先、beta 字符串次之排列：

$$I=I_\alpha N_\beta+I_\beta.$$

符号由 `cistring.des_sign` 求出。同一压缩自旋块内，固定的 alpha/beta 交叉相位会在 A_s 与 A_s^{T} 之间抵消。

全 CI 空间中，每块的非零连接数为

$$\mathrm{nnz}(A_s)=D\binom{n_\alpha}{m_\alpha}\binom{n_\beta}{m_\beta}.$$

这里仍需要访问 CI 系数和行列式连接；优化消除的是 D 次完整过渡 RDM，不是消除 CI 空间本身。

5 直接计算整个 CI 源项

固定权重时，每块辅助收缩为

$$\mathcal{F}_s = \sum_{PQJ} \widetilde{W}_{s,PQ} T_{s,PJ} T_{s,QJ}.$$

求导得到

$$Y_s = (\widetilde{W}_s + \widetilde{W}_s^{\mathsf{T}}) T_s, \tag{2}$$

$$\boldsymbol{b}^{\mathrm{CI,RDM}} = \sum_s A_s^{\mathsf{T}} \,\mathrm{vec}_{\mathrm{C}}(Y_s). \tag{3}$$

$\mathrm{vec}_{\mathrm{C}}$ 表示与 NumPy 的 C-order `ravel()` 一致的展开。若写成单个线性算符，式 (3) 等价于

$$A_s^{\mathsf{T}} \big[(\widetilde{W}_s + \widetilde{W}_s^{\mathsf{T}}) \otimes I_{R_s} \big] A_s c,$$

但代码不形成 Kronecker 积。

核心实现恰好是：

```
op, residual = _annihilator(norb, nelec, na, nb, tuples)
ket = (op @ ci.ravel()).reshape(count, residual)
compressed = _compressed_weight(weight, norb, rank, na, nb, tuples)
acted = (compressed + compressed.T) @ ket
out += op.T @ acted.ravel()
```

bra 和 ket 两侧的导数由 $\widetilde{W} + \widetilde{W}^{\mathsf{T}}$ 一起处理。

5.1 与 NEVPT2 代码的对应

- `grad/ci_contraction.py`：`_des_map` 生成连接与符号；`_annihilator` 组装 CSR；`_compressed_weight` 合并自旋和反对称排列；`contract_rdm_weights` 执行式 (3)。
- `grad/gradients.py`：`get_ci_response_legacy()` 完整保留旧循环；`get_ci_response_contracted()` 调用新收缩；`get_ci_response()` 根据后端分派并记录统计。
- `grad/dfgradients.py`：保留 DF 原有旧循环，复用相同的新源项接口；DF 积分和辅助基响应的其余部分未改变。

得到 CI 源项后，原程序继续追加其余源项并做归一化投影，再与轨道部分组成耦合 Z-vector 方程。没有提前改变投影顺序。

6 如何切换并保留旧实现

新默认后端为 `contracted`，旧路径仍可显式调用。对相同已收敛参考态，分别构造新的 PT/梯度对象：

```
pt_old = NEVPT2(mc, ...)
pt_old.kernel()
g_old = pt_old.nuc_grad_method()
```

```
g_old.ci_response_backend = 'legacy'
gradient_old = g_old.kernel()

pt_new = NEVPT2(mc, ...)
pt_new.kernel()
g_new = pt_new.nuc_grad_method()
g_new.ci_response_backend = 'contracted'
gradient_new = g_new.kernel()
print(g_new.ci_response_stats)
```

不要在同一个已累积过中间量的梯度对象上重复运行完整 kernel 来计时。常规积分和 DF 梯度都支持该开关。

7 测试条件和结果一致性

测试在 Intel Core i7-10700、约 16 GB 内存的本机执行，OpenMP/OpenBLAS/MKL 均固定为一个线程。每组新旧后端共享同一已收敛参考态，使用新的 PT 和梯度对象，先计算能量再计时梯度；三次重复并交替执行顺序，报告中位数。辅助模块在计时前导入；没有把剖析开销计入普通时间。

常规积分参考使用 Newton-CASSCF；DF 参考使用标准 DF-CASSCF，辅助基为 `weigend`，包含辅助基响应。单态参考收敛后包装为 NEVPT2 所需的单根容器，没有额外优化其他态；双态平均使用等权重。测试工具包含当前 PySCF 版本的单根 list/array 形状适配，与本次源项代数修改分开。

表 1: 算例。CAS 写成（电子数，轨道数）；目标根从 0 开始。

编号	分子/基组	CAS	D	积分	根数	目标根	canonstep
C1	LiH/STO-3G	$(2,2)$	4	ERI	1	0	0
C2	H_2O/STO-3G	$(2,2)$	4	ERI	1	0	0
C3	LiH/STO-3G	$(4,4)$	36	ERI	1	0	0
C4	LiH/STO-3G	$(4,5)$	100	ERI	1	0	0
C5	LiH/6-31G	$(4,6)$	225	ERI	1	0	0
C6	H_2O/STO-3G	$(2,2)$	4	DF	1	0	0
C7	LiH/STO-3G	$(4,5)$	100	DF	1	0	0
C8	LiH/STO-3G	$(2,2)$	4	ERI	2	0	2
C9	LiH/STO-3G	$(2,2)$	4	ERI	2	1	2
C10	LiH/STO-3G	$(2,2)$	4	ERI	1	0	1

对全部十组测试，前后最大差异为：

$$\begin{aligned} |\Delta E|_{\max} &= 0, \\ \|\Delta b^{\mathrm{CI,RDM}}\|_\infty &= 5.004 \times 10^{-15}, \\ \|\Delta b_{\mathrm{full}}\|_\infty &= 5.004 \times 10^{-15}, \\ \|\Delta Z\|_\infty &= 1.306 \times 10^{-14}, \\ \|\Delta \nabla E\|_\infty &= 6.211 \times 10^{-16} \quad \text{Hartree/Bohr}. \end{aligned}$$

核梯度前后比较覆盖每个算例的全部坐标分量。此外每个算例对末原子 z 坐标做全电子总能中心差分抽查：

$$g_{\mathrm{num}} = \frac{E(R+h) - E(R-h)}{2h}, \qquad h = 5 \times 10^{-4}\ \text{Bohr}.$$

每个位移点重新收敛参考态和 NEVPT2，包含核排斥、轨道和 CI 的变化。两种后端的最大抽查误差均不超过 9.031×10^{-8} Hartree/Bohr。这是代表坐标的差分抽查，不应称为所有坐标的数值差分验证。

7.1 前后耗时

表 2: 三次计时中位数，时间单位为秒。CI 源项时间和完整梯度时间分别报告。

编号	CI 旧	CI 新	CI 加速	梯度旧	梯度新	梯度加速
C1	0.004739	0.001086	4.4	0.1759	0.1605	1.10
C2	0.004839	0.001097	4.4	0.1790	0.1768	1.01
C3	0.266768	0.005605	47.6	0.3571	0.0969	3.68
C4	1.805743	0.011789	153.2	2.0010	0.1915	10.45
C5	11.347608	0.032685	347.2	11.8778	0.5505	21.58
C6	0.004679	0.001079	4.3	0.2303	0.2148	1.07
C7	1.817032	0.011385	159.6	2.0382	0.2315	8.80
C8	0.004765	0.001091	4.4	0.1690	0.1675	1.01
C9	0.004693	0.001070	4.4	0.1715	0.1672	1.03
C10	0.004733	0.001104	4.3	0.1781	0.1621	1.10

对 C5（LiH/CAS(4,6)，225 CI 分量），CI 源项由约 11.348 秒降到 0.0327 秒，加速约 347 倍；完整梯度由 11.878 秒降到 0.5505 秒，加速约 21.6 倍。旧路径对该源项构造 225 次过渡高阶 RDM，新路径为零次。仍需一次正常前向 RDM；Z-vector 内部也仍有低阶过渡密度计算。

7.2 为什么完整梯度加速小于源项加速

若源项占原总时间的比例为 f，源项加速为 S_{CI}，则近似有

$$S_{\mathrm{total}} \approx \frac{1}{(1-f) + f/S_{\mathrm{CI}}}.$$

C5 中 $f \approx 0.955$，该估计给出约 21 倍，与实测约 21.6 倍相符。小 CAS 中该源项只占几个百分点，因此即使源项加速数倍，完整梯度也可能只略有变化。缓存、库初始化和小样本计时波动也会影响小算例。

8 完整结构优化前后对照

同样采用 geomeTRIC、相同初始构型、相同参考态更新及收敛阈值，两种后端分别真正运行到收敛：最终坐标最大分量差异分别约为 5.05×10^{-12} 和 2.04×10^{-11} Bohr；终点能量差异分别约为

表 3: 优化总墙钟时间包含参考态更新、PT 能量、梯度和优化器开销。

体系	梯度评估次数旧/新	旧时间/秒	新时间/秒	加速
H_2O/STO-3G, CAS(2,2)	5/5	1.6089	1.5471	1.04
LiH/STO-3G, CAS(4,5)	4/4	8.5556	1.3072	6.54

2.84×10^{-14} 和 2.66×10^{-15} Hartree。这些差异处在本测试的浮点误差范围内，不代表引入了新的电子结构近似。

9 时间和内存标度仍有哪些限制

新源项的主要成本可按自旋块写成

$$T_{\mathrm{new}} \sim \sum_{s} \left[O(\mathrm{nnz}\, A_s) + O(M_s^2 R_s) \right] + T_{\mathrm{compress}}(a).$$

这消除了逐 CI 单位向量的完整过渡 RDM 调用，但 CI 维数仍是

$$D = \binom{a}{n_\alpha}\binom{a}{n_\beta},$$

半填充时仍有组合增长。权重和正常前向 4-RDM 的 a^8 存储、活性收缩块、CASSCF 与 Z-vector 迭代、外部积分和 AO 有效密度仍可能成为瓶颈。

一个源项自旋块的存储约为

$$O(\mathrm{nnz}\, A_s + M_s R_s + M_s^2),$$

并且一次只处理一个块。不能把旧算法的峰值内存误写成 D 个 4-RDM 同时驻留，也不能把此次加速解读为消除了所有大 CAS 或大分子限制。

10 复现、回归与适用边界

1. `tests/test_ci_response.py` 对常规/DF 两个梯度类，在随机 RDM 权重和不同 alpha/beta 电子数下比较新旧源项；禁止新路径调用过渡 4-RDM，检查内存预算和错误后端处理。这些不同自旋的单元测试不等于开壳层分子全部梯度路径验证。

2. 成对基准保存源项、完整右端、Z-vector、核梯度、耗时和正负位移能量；源文件的归一化 LF SHA256 也记录在每份 JSON 中。

3. 三次重复按新旧交替顺序串行执行，避免把不同机器、不同参考态或不相同的计算设置直接相除。峰值 RSS 为整个进程的峰值，不能据此单独归因每个后端的增量内存。

4. 单根容器兼容适配、严格为空的激发类别筛选均位于基准工具中，没有混入本次 NEVPT2 源项修改。

5. 验证包括常规积分、DF、canonstep=0/1/2、等权双态平均的两个目标根。没有据此声称验证了所有版本、复数波函数或所有开壳层分子。

示例命令（在本机 PySCF 工作区运行，按实际安装环境调整路径）：

```
export PYTHONPATH="$PWD"
export OMP_NUM_THREADS=1 OPENBLAS_NUM_THREADS=1 MKL_NUM_THREADS=1
python tools/benchmark_nevpt_ci_response.py \
  --nevpt-path ../nevpt_grad/embed_sim \
  --case lih_631g_e4a6 --repeats 3 --fd-check
python tools/run_nevpt_ci_response_benchmarks.py \
  --nevpt-path ../nevpt_grad/embed_sim
python tools/run_nevpt_ci_response_benchmarks.py \
  --nevpt-path ../nevpt_grad/embed_sim --optimization-only
```

11 学习时建议核对的三件事

第一，确认权重对应的是正常序、创建/湮灭指标分组后的 RDM；NEVPT2 与通用 PySCF 的 1-RDM 返回约定可能相差转置，这里使用实数波函数并同时处理 bra/ket 导数，必须通过旧实现逐项核验。第二，明确组合压缩的排列符号已经吸收到权重，不能再添加不一致的阶乘。第三，把原始 CI 源项、完整投影右端和最终核梯度分层比较；仅比较最终优化结构接近，不足以证明导数实现正确。